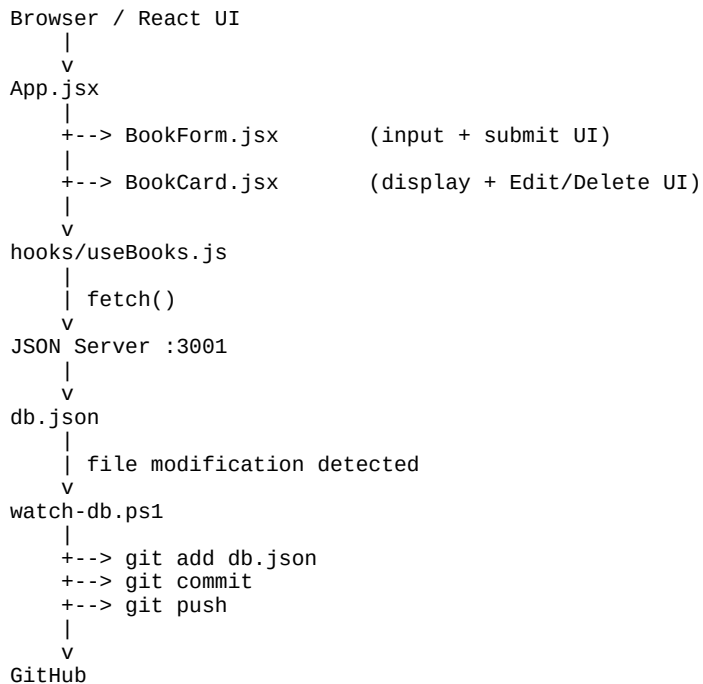


Book Library — Complete Flow & Learning Guide

React + JSON Server + db.json + Automatic Git Commit/Push

A repository-ready learning document explaining how the current project works from the UI all the way to the API, database file, and Git automation.

1. Project Architecture



2. The Main Data Flow

Initial load: App calls `useBooks()`. The hook's `useEffect` runs `loadBooks()`, sends GET /books, receives JSON, and stores it with `setBooks(data)`. App receives the books from the hook and passes each book to BookCard.

```
GET flow
App
-> useBooks()
-> useEffect()
-> loadBooks()
-> fetch(API_URL)
-> JSON Server GET /books
-> db.json
-> response.json()
-> setBooks(data)
-> App re-renders
-> BookCard receives book prop
```

3. CRUD Flow

Create

```
BookForm
-> handleSubmit()
-> event.preventDefault()
-> build bookData
-> onAdd(bookData)
-> addBook(bookData)
-> POST /books
-> JSON Server updates db.json
```

```
-> createdBook returned
-> setBooks(previousBooks => [...previousBooks, createdBook])
-> UI updates
```

Update

```
BookCard Edit
-> onEdit(book)
-> startEditing(book)
-> setEditingBook(book)
-> BookForm receives editingBook
-> user changes form
-> handleSubmit()
-> onUpdate(editingBook.id, bookData)
-> updateBook(id, bookData)
-> PUT /books/:id
-> JSON Server updates db.json
-> updatedBook returned
-> setBooks(previousBooks =>
  previousBooks.map(book =>
    book.id === updatedBook.id ? updatedBook : book
  )
)
-> UI updates
```

Delete

```
BookCard
-> onDelete(book.id)
-> deleteBook(id)
-> DELETE /books/:id
-> JSON Server updates db.json
-> setBooks(previousBooks =>
  previousBooks.filter(book => book.id !== id)
)
-> UI updates
```

4. App.jsx — Its Role

App.jsx is the coordinator between the data hook and the UI components. It does not contain the actual API implementation. It receives books and CRUD functions from useBooks(), manages which book is being edited, and passes data/functions to BookForm and BookCard.

```
import { useState, useRef } from "react";
import useBooks from "../hooks/useBooks";
import BookCard from "../components/BookCard";
import BookForm from "../components/BookForm";

function App() {
  const {
    books,
    loading,
    error,
    saving,
    addBook,
    updateBook,
    deleteBook,
  } = useBooks();

  const [editingBook, setEditingBook] = useState(null);
  const formRef = useRef(null);

  const startEditing = (book) => {
    setEditingBook(book);

    setTimeout(() => {
      formRef.current?.scrollIntoView({
        behavior: "smooth",
        block: "start",
      });
    }, 0);
  };

  const cancelEditing = () => {
    setEditingBook(null);
  };

  if (loading) {
    return <p>Loading...</p>;
  }

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100">
      <main className="mx-auto max-w-6xl px-6 py-10">
        <h1 className="mb-8 text-4xl font-bold">
          Book Library
        </h1>

        {error && (
          <p className="mb-5 rounded-lg border border-red-900 bg-red-950 p-4 text-red-300">
            {error}
          </p>
        )}

        <div ref={formRef}>
          <BookForm
            key={editingBook?.id || "new"}
            onAdd={addBook}
            onUpdate={updateBook}
            editingBook={editingBook}
            onCancelEdit={cancelEditing}
            saving={saving}
          />
        </div>

        {books.length === 0 ? (
          <p className="mt-8">No books found.</p>
        ) : (
          <div className="mt-8 grid gap-5 sm:grid-cols-2 lg:grid-cols-3">
            {books.map((book) => (
              <BookCard
                key={book.id}
                book={book}
                onEdit={startEditing}
                onDelete={deleteBook}
              />
            ))}
          </div>
        )}
      </main>
    </div>
  );
}
```

```

        </main>
      </div>
    );
  }
}

export default App;

```

5. Understanding the useBooks() Destructuring

```

const {
  books,
  loading,
  error,
  saving,
  addBook,
  updateBook,
  deleteBook,
} = useBooks();

```

useBooks() returns an object. The destructuring syntax takes the properties out of that returned object. App therefore gets both state (books, loading, error, saving) and functions (addBook, updateBook, deleteBook).

6. useBooks.js — Complete CRUD Logic

```

import { useEffect, useState } from "react";

const API_URL = "http://localhost:3001/books";

function useBooks() {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [saving, setSaving] = useState(false);

  // M1 - GET BOOKS
  useEffect(() => {
    async function loadBooks() {
      try {
        const response = await fetch(API_URL);

        if (!response.ok) {
          throw new Error("Failed to load books");
        }

        const data = await response.json();
        setBooks(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadBooks();
  }, []);

  // M2 - CREATE BOOK
  const addBook = async (bookData) => {
    setSaving(true);
    setError(null);

    try {
      const response = await fetch(API_URL, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(bookData),
      });

      if (!response.ok) {
        throw new Error("Failed to create book");
      }

      const createdBook = await response.json();

      setBooks((previousBooks) => [
        ...previousBooks,
        createdBook,
      ]);
    }
  };

```

```

    } catch (err) {
      setError(err.message);
    } finally {
      setSaving(false);
    }
  };

  // M3 - UPDATE BOOK
  const updateBook = async (id, updatedData) => {
    setSaving(true);
    setError(null);

    try {
      const response = await fetch(`${API_URL}/${id}`, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(updatedData),
      });

      if (!response.ok) {
        throw new Error("Failed to update book");
      }

      const updatedBook = await response.json();

      setBooks((previousBooks) =>
        previousBooks.map((book) =>
          book.id === updatedBook.id
            ? updatedBook
            : book
        )
      );
    } catch (err) {
      setError(err.message);
    } finally {
      setSaving(false);
    }
  };

  // DELETE BOOK
  const deleteBook = async (id) => {
    const confirmed = window.confirm(
      "Are you sure you want to delete this book?"
    );

    if (!confirmed) {
      return;
    }

    setError(null);

    try {
      const response = await fetch(`${API_URL}/${id}`, {
        method: "DELETE",
      });

      if (!response.ok) {
        throw new Error("Failed to delete book");
      }

      setBooks((previousBooks) =>
        previousBooks.filter((book) => book.id !== id)
      );
    } catch (err) {
      setError(err.message);
    }
  };

  return {
    books,
    loading,
    error,
    saving,
    addBook,
    updateBook,
    deleteBook,
  };
}

export default useBooks;

```

7. BookForm.jsx — Form Data Flow

```
function BookForm({
  onAdd,
  onUpdate,
  editingBook,
  onCancelEdit,
  saving,
}) {
  const [formData, setFormData] = useState({
    title: editingBook?.title || "",
    author: editingBook?.author || "",
    genre: editingBook?.genre || "",
    status: editingBook?.status || "want to read",
    rating: editingBook?.rating || 0,
  });

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData((previousData) => ({
      ...previousData,
      [name]: value,
    }));
  };

  const handleSubmit = async (event) => {
    event.preventDefault();

    const bookData = {
      title: formData.title,
      author: formData.author,
      genre: formData.genre,
      status: formData.status,
      rating: Number(formData.rating),
    };

    if (editingBook) {
      await onUpdate(editingBook.id, bookData);
    } else {
      await onAdd(bookData);
    }
  };

  // JSX contains:
  // <form onSubmit={handleSubmit}>
  //   <button type="submit"
  //     disabled={saving}
  //
```

Why `event.preventDefault()`? A normal HTML form has a browser default submission behavior. React needs to handle the submission itself, so `preventDefault()` cancels the browser's default action before the API logic runs.

Why `[name]: value`? `[name]` is a computed property name. If `name` is 'title', it becomes `title: value`. This lets one `handleChange` function update every input.

Why `disabled={saving}`? While POST/PUT is running, `saving` is true. The button is disabled to prevent repeated submissions.

8. Add vs Update Mode

```
editingBook = null
  -> Add mode
  -> onAdd(bookData)
  -> button shows "Add Book"

editingBook = a book object
  -> Edit mode
  -> onUpdate(editingBook.id, bookData)
  -> button shows "Update Book"

key={editingBook?.id || "new"}
  -> React component identity
  -> "new" in Add mode
  -> book id in Edit mode
```

The key does not directly decide the button text. `editingBook` does. The key helps React treat a different form instance as a different component when the identity changes.

9. BookCard.jsx — Props

```
function BookCard({ book, onEdit, onDelete }) {  
  const statusClass = {  
    "want to read": "bg-slate-800 text-slate-300",  
    reading: "bg-blue-950 text-blue-300",  
    finished: "bg-emerald-950 text-emerald-300",  
  };  
  
  // UI uses:  
  // book.title, book.author, book.genre, book.status, book.rating  
  // onEdit(book)  
  // onDelete(book.id)  
}
```

`book` comes from `books.map()` in `App`. `onEdit` is `App`'s `startEditing` function. `onDelete` is the `deleteBook` function returned by `useBooks()`.

The quotes around "want to read" are required because the property name contains spaces. `reading` and `finished` are valid unquoted JavaScript property names.

10. useRef and Smooth Scroll

```
const formRef = useRef(null);

const startEditing = (book) => {
  setEditingBook(book);

  setTimeout(() => {
    formRef.current?.scrollIntoView({
      behavior: "smooth",
      block: "start",
    });
  }, 0);
};

<div ref={formRef}>
  <BookForm ... />
</div>
```

useRef(null): The ref starts with no DOM element reference. React later places the actual div in formRef.current. The optional chaining operator (?.) safely calls scrollIntoView only if the reference exists.

setTimeout(..., 0): The callback is queued to run after the current JavaScript work, giving the React/browser update a chance to complete before the scroll is attempted.

11. JSON Server

```
Index:
http://localhost:3001/

Endpoints:
http://localhost:3001/books
```

The /books URL is an API endpoint, not a React Router route. The frontend can remain at localhost:5173/ without BrowserRouter because this project uses a single-page UI.

```
GET    /books      -> read all books
POST   /books      -> create a book
PUT    /books/:id  -> replace/update a book
DELETE /books/:id  -> delete a book
```

12. BrowserRouter vs preventDefault

BrowserRouter controls frontend URL-based navigation between React routes. This project does not need it because Add/Edit/Delete happen on one page.

event.preventDefault() controls the browser's default behavior for the form-submit event. These are independent concepts.

13. Why the Browser Was Reloading

In this project, JSON Server modifies db.json during POST/PUT/DELETE. Vite can notice project-file changes and trigger a browser reload. That is different from a normal React state re-render.

```
CRUD action
-> JSON Server
-> db.json changes
-> Vite sees project file change
-> browser reload may occur

React state update
-> setBooks(...)
-> React re-renders UI
-> no browser navigation required
```

event.preventDefault() prevents the browser's normal form submission; it does not prevent Vite from reacting to a file change made by JSON Server.

14. Automatic Git Watcher — watch-db.ps1

```
$projectPath = $PSScriptRoot
$dbFile = Join-Path $projectPath "db.json"

Write-Host "Watching db.json..."
Write-Host "Automatic Git commit + push enabled."
Write-Host "Press Ctrl+C to stop."

$lastWriteTime = (Get-Item $dbFile).LastWriteTime

while ($true) {

    Start-Sleep -Milliseconds 500

    $currentWriteTime = (Get-Item $dbFile).LastWriteTime

    if ($currentWriteTime -ne $lastWriteTime) {

        $lastWriteTime = $currentWriteTime

        Write-Host ""
        Write-Host "db.json changed. Updating Git..."

        Start-Sleep -Seconds 1

        Set-Location $projectPath

        git add db.json

        git commit -m "Auto update book data"

        git push

        Write-Host "Git update completed."

    }
}
```

15. Understanding the Watcher Code

\$projectPath = \$PSScriptRoot — finds the folder containing the script.

\$dbFile = Join-Path \$projectPath "db.json" — builds the full path to db.json.

\$lastWriteTime = (Get-Item \$dbFile).LastWriteTime — records when db.json was last changed as the starting reference.

while (\$true) — an infinite loop. \$true is PowerShell's Boolean true. The loop keeps checking until Ctrl+C stops the script.

\$currentWriteTime — gets the file's current modification time.

-ne — means 'not equal'. If the current time differs from the previous time, the file changed.

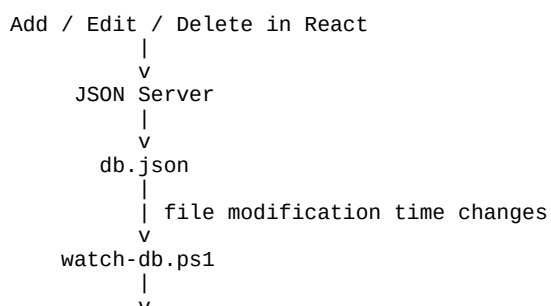
\$lastWriteTime = \$currentWriteTime — remembers the new time so the same change is not processed repeatedly.

git add db.json — stages the changed database file.

git commit -m ... — creates a local Git commit.

git push — sends the local commit to the configured remote repository such as GitHub.

16. Automatic Git Flow



```

git add db.json
  |
  v
git commit -m "Auto update book data"
  |
  v
git push
  |
  v
GitHub

```

17. Running the Project

Terminal 1:
npm run dev

Terminal 2:
npx json-server db.json --port 3001

Terminal 3:
powershell -ExecutionPolicy Bypass -File .\watch-db.ps1

Keep the watcher terminal running. It is designed to continue watching after each Add/Edit/Delete. Stop it manually with Ctrl+C.

18. Final Mental Model

```

UI responsibility:
BookForm / BookCard
  |
  v
App responsibility:
connect UI to functions/state
  |
  v
Data/API responsibility:
useBooks.js
  |
  v
Backend responsibility:
JSON Server
  |
  v
Storage:
db.json
  |
  v
Version-control automation:
watch-db.ps1
  |
  v
Git / GitHub

```

Remember this sentence: React changes the data through the API; JSON Server writes db.json; the PowerShell watcher notices db.json changed; Git commits it and pushes it to GitHub.

19. Quick Reference

Concept	Meaning in this project
useState	Stores changing React state such as books, editingBook, saving
useEffect	Runs the initial GET request after component rendering
useRef	Stores a DOM reference for scrolling to the form
Props	Pass data/functions from App to BookForm or BookCard
preventDefault	Stops the browser's default form submission behavior
key	Gives React component identity; changes can cause a fresh form instance
fetch	Sends HTTP requests to JSON Server
JSON Server	Provides REST endpoints backed by db.json

Concept	Meaning in this project
LastWriteTime	File modification time used by the watcher
while (\$true)	Keeps the PowerShell watcher running continuously
-ne	PowerShell 'not equal' comparison
git add	Stages changes
git commit	Creates a local Git snapshot
git push	Sends commits to the remote repository